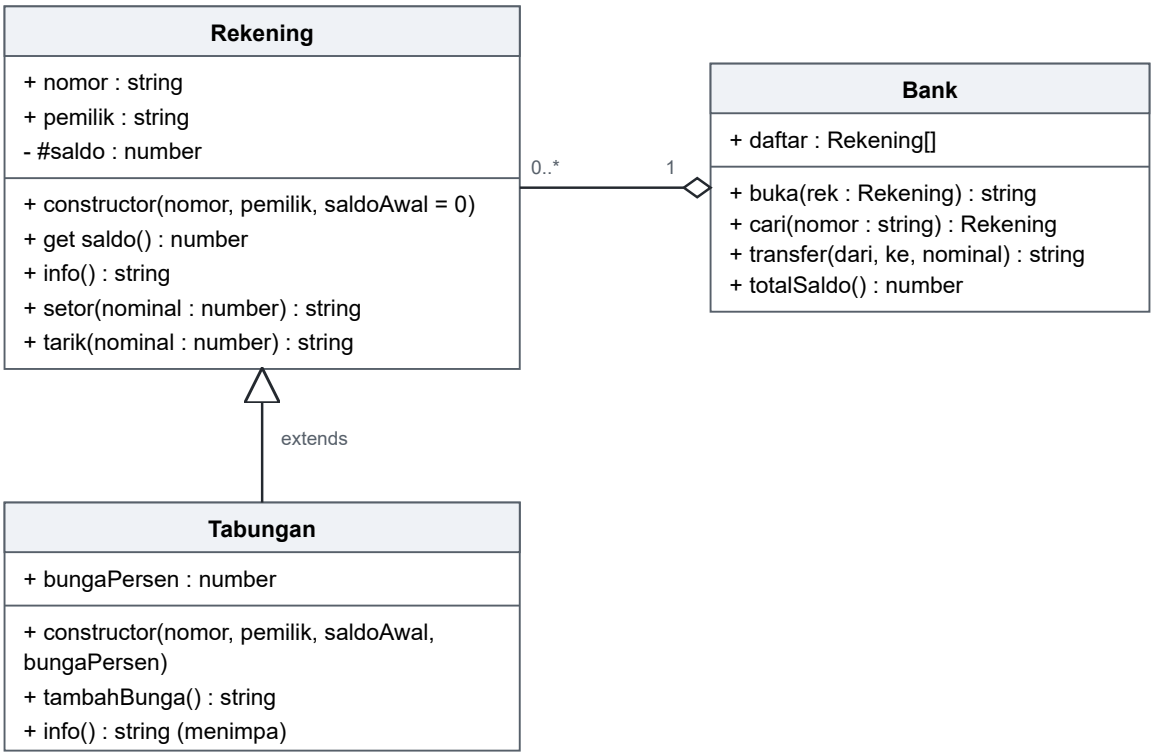
Praktikum JavaScript & TypeScript

Soal Modul 5 - Object-Oriented Programming

1. Tugas

Kamu membuat sistem rekening untuk sebuah bank kecil. Ada tiga class: `Rekening`, `Tabungan` (turunan `Rekening` yang punya bunga), dan `Bank` yang menyimpan banyak rekening.



Gambar 1: UML class diagram. Tanda `+` berarti publik, `-` berarti privat. Panah segitiga berarti `extends`. Belah ketupat berarti `Bank` menyimpan nol atau banyak `Rekening`.

> **⚠ Perhatian**
>
> 1. Salin `Soal/starter.js` ke folder `Jawaban/`, misalnya `Jawaban/soal-js5.js`.
> 2. Lengkapi bagian bertanda `TODO`. Jangan mengubah bagian `PEMANGGILAN`.
> 3. Jalankan `node Jawaban/soal-js5.js`. Kotak **Output dari** `console.log(...)` di bawah setiap tugas menunjukkan baris `console.log` yang mencetaknya dan output yang harus keluar persis.
> 4. Soal bertanda **(jelaskan)** dijawab sebagai komentar di bawah baris `// JAWABAN ...`.
> 5. Pemanggilan berjalan berurutan dan memakai object yang sama, jadi saldo di satu tugas terbawa ke tugas berikutnya. Semua number adalah bilangan bulat.

1.1. Beginner (0.3)

1. **Class `Rekening`.**
   - `constructor(nomor, pemilik, saldoAwal = 0)`. Input: `nomor` string, `pemilik` string, `saldoAwal` number.
   - `nomor` dan `pemilik` disimpan sebagai field publik. Saldo disimpan di field **privat** `#saldo`.
   - `get saldo()`. Output: number, isi `#saldo`. Saldo hanya bisa dibaca dari luar, tidak bisa diubah langsung.

- `info()`. Output: string format `<nomor> a.n. <pemilik>: saldo <saldo>`.
- Contoh:
  - `new Rekening("1001", "Ani", 500000)` menghasilkan object dengan `nomor = "1001"`, `pemilik = "Ani"`, dan `#saldo = 500000`.
  - `new Rekening("3001", "Cici")` tanpa argumen ketiga menghasilkan `#saldo = 0`.
  - Untuk object pertama, `info()` menghasilkan `"1001 a.n. Ani: saldo 500000"`, dan `saldo` bernilai `500000`.

Output dari `const ani = new Rekening("1001", "Ani", 500000);` lalu `console.log(ani.info())`

```
1001 a.n. Ani: saldo 500000
```

Output dari `console.log(ani.saldo)`

```
500000
```

2. **Method `setor(nominal)`.**
   - Input: `nominal` number. Output: string.
   - Jika `nominal` 0 atau negatif: kembalikan `GAGAL: nominal harus lebih dari 0`, saldo tidak berubah.
   - Selain itu: tambahkan ke saldo, kembalikan `Setor <nominal>, saldo <saldoBaru>`.
   - Contoh (saldo `ani` sebelumnya `500000`):
     - `ani.setor(250000)` menghasilkan `"Setor 250000, saldo 750000"`. Saldo `ani` menjadi `750000`.
     - `ani.setor(0)` menghasilkan `"GAGAL: nominal harus lebih dari 0"`. Saldo tetap `750000`.

Output dari `console.log(ani.setor(250000))`

```
Setor 250000, saldo 750000
```

Output dari `console.log(ani.setor(0))`

```
GAGAL: nominal harus lebih dari 0
```

3. **Method `tarik(nominal)`.**
   - Input: `nominal` number. Output: string.
   - Cek berurutan, kembalikan pesan pertama yang cocok:
     a. `nominal` 0 atau negatif: `GAGAL: nominal harus lebih dari 0`
     b. `nominal` lebih besar dari saldo: `GAGAL: saldo tidak cukup (sisa <saldo>)`
   - Jika lolos: kurangi saldo, kembalikan `Tarik <nominal>, saldo <saldoBaru>`.
   - Contoh (saldo `ani` sebelumnya `750000`, dari Beginner 2):
     - `ani.tarik(100000)` menghasilkan `"Tarik 100000, saldo 650000"`. Saldo `ani` menjadi `650000`.
     - `ani.tarik(900000)`: `900000` lebih besar dari `650000`, jadi hasilnya `"GAGAL: saldo tidak cukup (sisa 650000)"`. Saldo tetap.

Output dari `console.log(ani.tarik(100000))`

```
Tarik 100000, saldo 650000
```

Output dari `console.log(ani.tarik(900000))`

```
GAGAL: saldo tidak cukup (sisa 650000)
```

Output dari `console.log(ani.tarik(-5))`

```
GAGAL: nominal harus lebih dari 0
```

4. **(jelaskan)** Temanmu ingin “mengisi saldo cepat” dan menulis kode ini di luar class. Jalankan di `cobaSendiri()` atau file terpisah.

```
const r = new Rekening("9001", "Tes", 0);
r.saldo = 1000000;
```

   - Tulis pesan error lengkapnya.

- Mengapa saldo sengaja dibuat tidak bisa diubah langsung? Apa yang bisa terjadi di aplikasi bank jika `saldo` adalah field publik biasa?

1.2. Intermediate (0.3)

1. **Class `Tabungan`.**
   - `class Tabungan extends Rekening`.
   - `constructor(nomor, pemilik, saldoAwal, bungaPersen)`. Teruskan tiga parameter pertama ke `Rekening` dengan `super`. Simpan `bungaPersen` (number) sebagai field publik.
   - `tambahBunga()`. Output: string format `Bunga <bunga>, saldo <saldoBaru>`.
     - `<bunga>` : `bungaPersen` persen dari saldo, dibulatkan ke bawah.
     - Bunga masuk ke saldo. `#saldo` milik `Rekening` tidak bisa diakses dari `Tabungan`, jadi pakai method yang sudah ada.
   - Contoh:
     - `new Tabungan("2001", "Budi", 1000000, 2)` menghasilkan object dengan `nomor = "2001"`, `pemilik = "Budi"`, `#saldo = 1000000` (diisi lewat `super`), dan `bungaPersen = 2`.
     - `tambahBunga()` : 2% dari `1000000` adalah `20000`, jadi hasilnya `"Bunga 20000, saldo 1020000"`.
     - `budi.tarik(20000)` langsung jalan tanpa ditulis ulang di `Tabungan`, karena diwarisi dari `Rekening`.

   Output dari `const budi = new Tabungan("2001", "Budi", 1000000, 2);` lalu `console.log(budi.tambahBunga())`

   ```
   Bunga 20000, saldo 1020000
   ```

   Output dari `console.log(budi.tarik(20000))`

   ```
   Tarik 20000, saldo 1000000
   ```

2. **Menimpa `info()` di `Tabungan`.**
   - Output: string format `<infoRekening>, bunga <bungaPersen>%`. `<infoRekening>` adalah hasil `info()` milik `Rekening`.
   - Jangan menyalin isi `info()` milik `Rekening`. Panggil versi induknya.
   - `info()` milik `Rekening` sendiri tidak berubah.
   - Contoh (saldo `budi` sekarang `1000000`): `info()` induk menghasilkan `"2001 a.n. Budi: saldo 1000000"`. `info()` milik `Tabungan` menambahkan `", bunga 2%"` di belakangnya. `ani.info()` tetap memakai format lama.

   Output dari `console.log(budi.info())`

   ```
   2001 a.n. Budi: saldo 1000000, bunga 2%
   ```

   Output dari `console.log(ani.info())`

   ```
   1001 a.n. Ani: saldo 650000
   ```

3. **Kode temanmu: `setorBanyak(rek, daftarNominal)`.**
   - Output yang benar: number, saldo `rek` setelah semua nominal di `daftarNominal` disetor.
   - Contoh: `rek` adalah `new Rekening("3001", "Cici")` dengan saldo `0`, dan `daftarNominal` adalah `[50000, 20000, 30000]`. Ketiganya disetor satu per satu, jadi hasilnya `100000`.
   - Masalah: setelah `setor` di Beginner 2 selesai, fungsi ini melempar `TypeError: Cannot read properties of undefined (reading '#saldo')`.
   - **(jelaskan)** Method `setor` jalan normal jika dipanggil `rek.setor(50000)`. Mengapa error ketika `rek.setor` diberikan ke `forEach` sebagai callback? Lalu perbaiki.

   Output dari `console.log(setorBanyak(new Rekening("3001", "Cici"), [50000, 20000, 30000]))`

   ```
   100000
   ```

1.3. Advanced (0.4)

Advanced dinilai dari **pendekatannya**. Bagian (a) wajib dan menentukan nilai. Bagian (b) opsional. Kode yang kosong tidak mengurangi nilai.

1. **Class `Bank`.**
   - `constructor()` : field `daftar` berisi array kosong.
   - `buka(rek)` : masukkan `rek` ke `daftar`. Output: `Rekening <nomor> dibuka`.
   - `cari(nomor)` : Output: `Rekening` yang `nomor`-nya sama, atau `undefined`.
   - `transfer(dariNomor, keNomor, nominal)`. Output: string. Cek berurutan:
     a. rekening asal tidak ada: `GAGAL: rekening <dariNomor> tidak ditemukan`
     b. rekening tujuan tidak ada: `GAGAL: rekening <keNomor> tidak ditemukan`
     c. penarikan dari rekening asal gagal: kembalikan pesan `GAGAL` dari `tarik` apa adanya
     d. jika lolos, setor ke tujuan dan kembalikan `Transfer <nominal> dari <dariNomor> ke <keNomor> berhasil`
   - `totalSaldo()` : Output: number, jumlah saldo semua rekening.
   - Contoh (sebelum transfer, saldo `ani` `650000` dan saldo `budi` `1000000`):
     - Setelah `bank.buka(ani)` dan `bank.buka(budi)`, `daftar` berisi `[ani, budi]`.
     - `bank.cari("1001")` mengembalikan object `ani`. `bank.cari("9999")` mengembalikan `undefined`.
     - `bank.transfer("1001", "2001", 150000)` : saldo `ani` menjadi `500000`, saldo `budi` menjadi `1150000`.
     - `bank.totalSaldo()` : `500000 + 1150000` adalah `1650000`.

   a. **(jelaskan, wajib)** Tulis pendekatanmu di komentar `// JAWABAN Advanced 1a`. Untuk setiap method `Bank`, tulis langkahnya dan method array atau method `Rekening` yang dipakai. Jelaskan juga: mengapa `transfer` memanggil `tarik` dan `setor`, bukan mengubah saldo sendiri? Dan mengapa urutannya `tarik` dulu baru `setor`?
   b. **(opsional)** Tulis kodenya.

Output dari `const bank = new Bank();` lalu `console.log(bank.buka(ani))`

```
Rekening 1001 dibuka
```

Output dari `console.log(bank.buka(budi))`

```
Rekening 2001 dibuka
```

Output dari `console.log(bank.transfer("1001", "2001", 150000))`

```
Transfer 150000 dari 1001 ke 2001 berhasil
```

Output dari `console.log(bank.transfer("1001", "9999", 1000))`

```
GAGAL: rekening 9999 tidak ditemukan
```

Output dari `console.log(bank.transfer("1001", "2001", 999999))`

```
GAGAL: saldo tidak cukup (sisa 500000)
```

Output dari console.log(`${ani.saldo} ${budi.saldo} ${bank.totalSaldo()}`)

```
500000 1150000 1650000
```

2. Pengumpulan

⚠ Perhatian

- Kode soal: `Jawaban/soal-js5.js`. Jawaban **(jelaskan)** ditulis sebagai komentar di bawah baris `// JAWABAN ...` yang sesuai.

- Coba Sendiri dari HO dikerjakan di fungsi `cobaSendiri()` pada file yang sama. Jalankan dengan `node Jawaban/soal-js5.js coba`.
- Jalankan `npm run selesai -- --durasi "..." --macet "..."`, lalu minta asisten memeriksa.